

PPC Code Review — Fault Handling & Edge Cases

Repository: BucurCosmin/PPC

Reviewer: Claude (Anthropic)

Date: 2026-05-25

Focus: Inverter errors, comm errors, derating, disabled inverters, and recovery paths

Executive Summary

The overall architecture is well-designed — the fixed FC execution order, the separation between PPC logic and comms layer, and the use of a single inverter array passed as VAR_IN_OUT are all sound choices. The code is exceptionally well-documented.

However, I've identified **3 critical issues** (one will prevent compilation), **5 significant issues** that affect runtime fault behavior, and several minor observations. The most important finding is that `ErrClr` does not exist in your UDT, so the FaultHandler will not compile as written.

CRITICAL Issues

C1. `ErrClr` Field Missing from `Inverter_controller` UDT — Will Not Compile

Where: `FC_PPC_FaultHandler.scl`, lines 123, 135, 146, 160, 162

What: The FaultHandler writes to `Inverters[i].ErrClr` extensively (setting it to 0 or 26 for the acknowledge one-shot). However, the `Inverter_controller` UDT exported from TIA Portal (PDF in `PLC TYPES/`) does **not** contain an `ErrClr` field.

Impact: The code will fail to compile in TIA Portal. The entire fault acknowledgment sequence (S3.3 — writing `ErrClr=26` after fault clears) is non-functional.

Fix: Add `ErrClr : DInt` to the `Inverter_controller` UDT in TIA Portal. The comms block must then map this field to SMA Holding Register 8 (`ErrClr / FltRst`).

C2. Derating State (`DrtStt`) Is Read but Never Acted On

Where: All FCs — `DrtStt` exists in the UDT but no FC reads or uses it.

What: The `DrtStt` field is populated by the comms block (from SMA input register), but the PPC logic completely ignores it. A derated inverter (thermal limit, grid frequency derating, etc.) remains "online" and receives full proportional share of power demand.

Impact: This is functionally handled *indirectly* because `WAval` decreases when an inverter derates — the proportional distribution algorithm naturally gives a derated unit less power. **However**, there are problems:

1. **No HMI/SCADA alarm for derating.** The operator has no visibility that inverters are derated unless they manually check `WAval` values. A plant could be running at 70% capacity with zero alarms.
2. **No differentiation between "healthy but derated" and "fully available."** If SCADA needs to report plant availability to the grid operator, it can only report `PmaxPlant` (the `WAval` sum), but has no breakdown of *why* capacity is reduced.
3. **Rapid derating oscillation.** If an inverter rapidly oscillates between derated and full capacity (e.g., temperature cycling), `WAval` changes every scan cycle and power redistribution jitters — there's no damping or hysteresis on the availability signal.

Fix: Add `DrtStt` supervision in `InverterMonitor` or `FaultHandler`:

```
scl

// In FaultHandler, healthy inverter section:
IF #Inverters[#i].DrtStt <> 0 THEN
    #AnyDerating := TRUE; // New output flag for HMI
END_IF;
```

Consider adding a derating alarm output separate from `AnyFault`, and optionally a derating counter or bitmap so SCADA knows *which* inverters are derated.

C3. No Individual Inverter Fault Feedback to HMI/SCADA

Where: `FC_PPC_FaultHandler.scl` — only sets a single `AnyFault` BOOL.

What: The `FaultHandler` collapses all inverter fault information into one plant-level Boolean. There is no per-inverter fault status output, no fault code logging, and no indication of *which* inverter is faulted or *what* the fault is.

Impact: When `PPC_Fault` goes TRUE on the HMI, the operator must manually browse all 10 inverters to find which one faulted. In a real plant, this is a serious diagnostic limitation. The `ErrNo` register (SMA input register 96, fault code) is mentioned in comments (S3.2: "should be logged by SCADA") but there's no mechanism to surface it.

Fix: Add to the UDT or to a parallel diagnostics array:

- Per-inverter fault status bits (or a plant-level "FaultedInverterMask" as a Word/DWord)
- Capture and latch last `ErrNo` per inverter for HMI display
- Separate fault counters per inverter (optional, for trending)

SIGNIFICANT Issues

S1. Power Redistribution Lag When an Inverter Faults Mid-Cycle

Where: Execution chain ①→④→⑥

What: Consider the timing when inverter #3 faults between two OB30 scans:

1. **Scan N:** InverterMonitor (①) runs first — sees #3 as online (Error was FALSE at scan start). Counts it in N_Online and PmaxPlant.
2. **PowerDistribution (④):** Allocates a share of Ramps_Pcmd to #3 based on its WAvail.
3. **FaultHandler (⑥):** Now sees #3.Error = TRUE → zeros its WSpt.

The result: during scan N, the power that was allocated to #3 by PowerDistribution is zeroed by FaultHandler, but the other 9 inverters still only received their reduced shares. **The plant briefly under-delivers by the faulted inverter's share for one scan cycle (100ms).**

Impact: For a single scan (100ms), the plant output dips. On the next scan, InverterMonitor will exclude #3 and redistribute correctly. In practice, 100ms is likely acceptable for grid compliance, but for tight frequency response requirements it's worth noting.

Possible mitigation: After FaultHandler runs, do a second pass in the FB to check if any inverter was "demoted" during this scan and redistribute. Or accept the one-scan lag as a documented design limitation.

S2. `PwrOffReas` Sets `AnyFault` but Doesn't Trigger Any Protective Action

Where: `FC_PPC_FaultHandler.scl`, lines 174-176

What: When a healthy inverter has `PwrOffReas ≠ 0`, the code sets `AnyFault := TRUE` but takes no other action — the inverter continues to receive OperMode=308 and its full WSpt allocation.

Impact: PwrOffReas indicates the inverter disconnected for an abnormal reason (e.g., code 21626 = "Low Power SetPoint", or grid protection trips). An inverter reporting PwrOffReas ≠ 0 may not actually be producing power even though the PPC thinks it's online. This creates a phantom capacity problem — PPC allocates power to an inverter that isn't actually delivering.

Additionally: PwrOffReas is never cleared. If the comms block reads PwrOffReas once and it's non-zero, does the register auto-clear on the inverter side after restart? If not, AnyFault will remain TRUE permanently after the first PwrOffReas event.

Fix: Either:

- Treat PwrOffReas $\neq 0$ as equivalent to an Error condition (zero setpoints, exclude from online count), OR
 - Cross-check PwrOffReas against Wactive feedback — if PwrOffReas $\neq 0$ AND Wactive = 0, the inverter isn't producing and should be excluded from distribution, OR
 - At minimum, log PwrOffReas per inverter and add a separate alarm flag (not lumped into AnyFault)
-

S3. Comm Error During Active Production — Race Condition with Distribution

Where: `FC_PPC_PowerDistribution.scl` and `FC_PPC_FaultHandler.scl`

What: When CommError goes TRUE on an inverter that was carrying significant load:

1. PowerDistribution immediately excludes it (pass 1 sum excludes it, pass 2 gives it WSpt=0).
2. FaultHandler confirms the stop.
3. **But:** The remaining online inverters only get redistributed based on their share of Ramps_Pcmd — and Ramps_Pcmd hasn't changed yet. It's still the same value from before the fault.

This means: if you had 10 inverters sharing 10 MW and one drops out, the remaining 9 immediately receive $10 \text{ MW} / 9 \approx 1.11 \text{ MW}$ each. If each inverter was previously at 1.0 MW and WAval is only 1.05 MW, **some inverters may receive setpoints exceeding their WAval.**

Mitigation already present: RampControl clamps Ramps_Pcmd $\leq$ PmaxPlant. But PmaxPlant was computed by InverterMonitor **before** the CommError was detected (same scan issue as S1). So for one scan, Ramps_Pcmd may exceed the actual available capacity.

Impact: The inverter will clip internally (it won't produce more than WAval), but writing WSpt > WAval may trigger SMA warnings or PwrOffReas codes.

Fix: Consider a second PmaxPlant calculation after FaultHandler, or add per-inverter WSpt clamping:

```
scl

// After computing share-based WSpt, clamp to individual WAval
#maxWSpt := REAL_TO_DINT(DINT_TO_REAL(#Inverters[#i].WAval) / 10000.0 * #WRtg_kW);
#Inverters[#i].WSpt := MIN(IN1 := #Inverters[#i].WSpt, IN2 := #maxWSpt);
```

S4. FALLBACK Recovery: Instant Jump to REMOTE_IEC Without Stabilization

Where: `FC_PPC_ModeManager.scl`, line 75

What: When comms recover (WD_Expired goes FALSE), ModeManager immediately sets Plant_Mode = 1 (REMOTE_IEC). On the very next scan:

- RampControl uses the full Cmd_P as target (no longer overridden to 0)
- PowerDistribution writes setpoints
- FaultHandler writes OperMode=308

If Ramps_Pcmd was reset to 0 during FALLBACK (because onlineCount dropped to 0 when all inverters were stopped), the ramp will start from 0 and respect P_RampUp — this is correct.

BUT: If Ramps_Pcmd was NOT fully reset (some inverters stayed online during FALLBACK — possible if the FaultHandler stop command didn't fully take effect before comms restored), Ramps_Pcmd retains a stale non-zero value and the plant may jump.

Additionally: There's no "hold" period after FALLBACK recovery. In grid codes, some operators require a stabilization delay (e.g., 5-10 seconds of zero output) before ramping back. The current design ramps back immediately.

Fix: Consider adding a configurable recovery delay timer in ModeManager, and always resetting Ramps_Pcmd to 0 on FALLBACK→REMOTE transition (not only when onlineCount = 0).

S5. No Reactive Power Ramp Rate Limiting

Where: FC_PPC_ReactiveControl.scl

What: Q_Ramp and PF_Ramp exist in DB39 (from the PPC_Controller PDF), but ReactiveControl applies **no ramp logic** — it writes the target Q or PF directly to inverters every scan cycle. A large step change in Cmd_Q goes straight through to VArSpt with zero rate limiting.

Impact: Abrupt reactive power steps can cause voltage transients on the grid, especially at high power levels. Many grid codes require Q ramp rates.

Fix: Add ramp logic for Q similar to what RampControl does for P, or at minimum add a first-order filter on Targets_Q before distribution.

MINOR Issues

M1. LOCAL Mode Behavior Undefined

When Plant_Mode = 0 (LOCAL), the FB still runs the full chain (InverterMonitor → ModeManager → RampControl → PowerDistribution → ReactiveControl → FaultHandler). RampControl uses Cmd_P as the target — but in LOCAL mode, Cmd_P comes from the OB30 caller which gets it from the upstream comms thread. If the operator is in LOCAL, shouldn't targets come from the HMI instead?

The design doc mentions "Targets come from HMI/local DB" for LOCAL mode, but the code doesn't switch the target source. The caller (OB30) would need to handle this, and it's not documented how.

M2. Watchdog Edge Detection Only Detects Rising Edge (0→1)

The watchdog uses `(IEC_Watchdog AND NOT Watchdog_LastBit)` — this only detects 0→1 transitions. If the upstream comms thread toggles the bit (0→1→0→1), only every other transition is detected. If the comms thread sets it to TRUE and holds it (without toggling back to FALSE), only one edge is ever detected and the watchdog will eventually expire even though comms are healthy.

This is documented correctly in the code comments ("toggles once per received telegram"), but it's worth confirming with the comms thread developer that the bit actually toggles (goes 1→0→1) rather than just pulsing or latching.

M3. CASE Statement in ReactiveControl Has No ELSE Branch

`FC_PPC_ReactiveControl.scl`, line 104-136: The `CASE #Plant_VArMode OF` statement handles values 0, 1, and 2 but has no `ELSE` clause. If `Cmd_VArMode` from upstream contains an unexpected value (e.g., 3, or -1), no reactive setpoint is written at all — the inverter retains whatever VArSpt/PFSpt/VArMode was written in the previous scan.

Fix: Add an ELSE that defaults to mode 0 (Off):

```
scl

ELSE
    // Unknown mode – treat as Off for safety
    #Inverters[#i].VArSpt := 0;
    #Inverters[#i].PFSpt  := 0;
    #Inverters[#i].VArMode := 303;
```

M4. No Wactive/Qactive Feedback Validation

The UDT contains `Wactive` and `Qactive` (measured power feedback from inverters) but no FC uses them. There's no closed-loop verification that inverters are actually producing what they're commanded. This means:

- A stuck inverter (WSpt = 500 kW but Wactive = 0 kW) goes undetected

- An inverter producing more than commanded (runaway) goes undetected
- Plant-level power measurement at PCC (`Plant_P_meas` in DB39) is never compared against the sum of inverter setpoints

This isn't strictly a fault-handling bug, but it's a significant gap in supervision.

M5. FaultHandler Writes OperMode=303 to CommError Inverters — Futile

When CommError = TRUE, the comms link is broken. Writing `OperMode := 303` to the UDT is correct (it's the intended safe state), but the comms block can't actually send this to the inverter because the link is down. The code comment acknowledges this ("Best-effort — inverter's own internal watchdog will trip it"), which is the right approach. However, consider whether the comms block should:

- Retry the stop command when comms recover (before sending OperMode=308)
 - Or whether the inverter's own Modbus watchdog timeout is sufficient
-

M6. `Enabled` Flag Interaction with Fault Recovery

If an operator disables an inverter (`Enabled = FALSE`) while it has an active Error, and later re-enables it after the error clears, the `PrevError` falling-edge detection will have already fired (and been consumed) during the scan when Error went FALSE. When Enabled goes TRUE again, no ErrClr=26 one-shot will be sent because `PrevError[i]` was already updated.

This means: an inverter that was disabled during a fault and re-enabled after the fault cleared will never receive the ErrClr acknowledgment. It may remain in a latched-fault state on the SMA side, unable to restart.

Fix: When transitioning Enabled from FALSE→TRUE, force a one-shot ErrClr=26 write regardless of edge detection, or require the operator to explicitly acknowledge faults per inverter.

Summary Table

ID	Severity	Description	Action Required
C1	Critical	ErrClr field missing from UDT — won't compile	Add ErrClr : DInt to UDT
C2	Critical	Derating (DrtStt) ignored — no alarm, no visibility	Add derating supervision + HMI alarm

ID	Severity	Description	Action Required
C3	Critical	No per-inverter fault info to HMI — only AnyFault BOOL	Add per-inverter diagnostics
S1	Significant	1-scan power dip when inverter faults mid-cycle	Document or add 2nd redistribution pass
S2	Significant	PwrOffReas sets AnyFault but no protective action	Add exclusion logic or cross-check with Wactive
S3	Significant	CommError can cause WSpt > WAval for 1 scan	Add per-inverter WSpt clamp
S4	Significant	FALLBACK→REMOTE recovery may be too aggressive	Add recovery delay; force Ramps_Pcmd=0 on transition
S5	Significant	No Q/PF ramp rate limiting despite Q_Ramp in DB	Implement reactive ramp logic
M1	Minor	LOCAL mode target source undefined in code	Clarify OB30 caller responsibility
M2	Minor	Watchdog assumes toggling bit — verify comms behavior	Confirm with comms developer
M3	Minor	CASE statement missing ELSE for invalid VArMode	Add ELSE → mode Off
M4	Minor	No Wactive/Qactive feedback loop — no production validation	Consider adding setpoint vs. feedback check
M5	Minor	Stop command to CommError inverter is futile	Document reliance on SMA watchdog
M6	Minor	Disabled-during-fault inverter misses ErrClr on re-enable	Add Enabled rising-edge ErrClr logic